

Estruturas Essenciais - Linguagem C

Apresentadores



Enzo



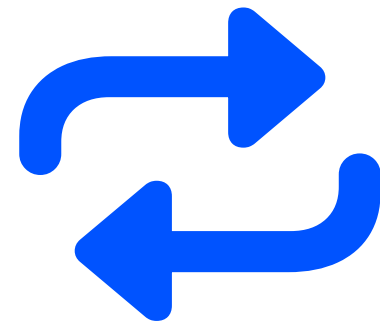
Leonel



Késia

Agenda

Agenda



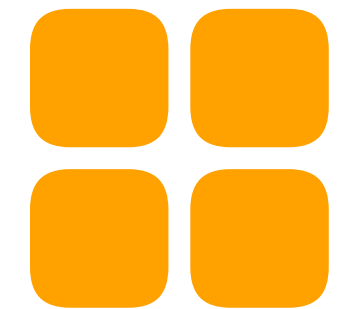
**Laços de
repetição**



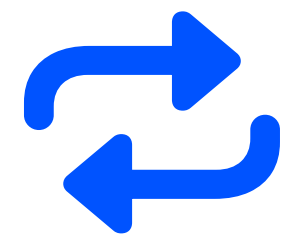
Vetores

$f(x)$

Funções



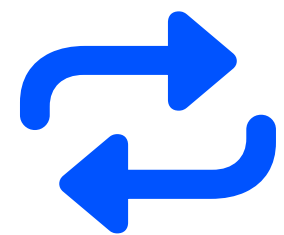
Structs



Laços de repetição

O que são?

Os laços de repetição são comandos de programação que permitem executar um bloco de código várias vezes de forma automática, evitando a necessidade de repetir instruções manualmente.



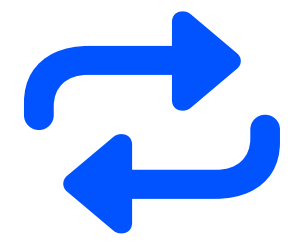
Laços de repetição

Sem laço de
repetição

```
escreva("oi")  
escreva("oi")  
escreva("oi")  
escreva("oi")  
escreva("oi")
```

Com laço de
repetição

```
repita 5 vezes:  
    escreva("oi")
```



Laços de repetição

Ganhos no uso deles:

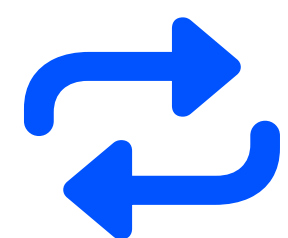
Menos código

Manutenção mais fácil

Escalabilidade

Legibilidade

Flexibilidade

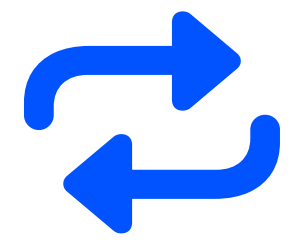


Laços de repetição

While

Do-While

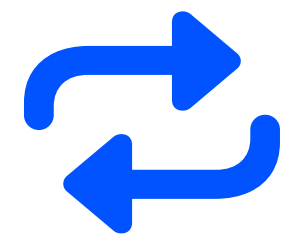
For



Laços de repetição

While

Executa o bloco de código **enquanto** uma condição for verdadeira.

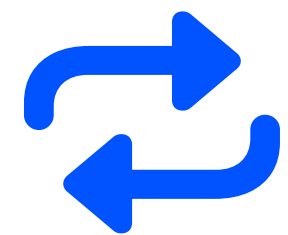


Laços de repetição

While

```
int vidas = 3;

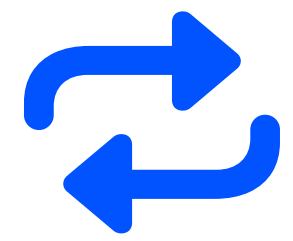
while (vidas > 0) {
    printf("Vidas restantes: %d\n", vidas);
    vidas--;
}
```



Laços de repetição

Do-While

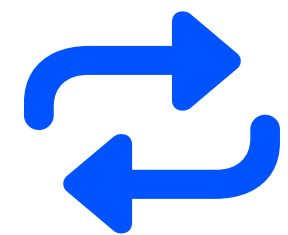
Executa o bloco de código **pelo menos uma vez** e só depois verifica a condição.



Laços de repetição

Do-While

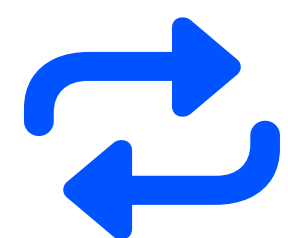
```
int senha;  
  
do {  
    printf("Digite a senha: ");  
    scanf("%d", &senha);  
} while (senha != 123);
```



Laços de repetição

For

Executa o bloco de código um **número determinado de vezes** com base em um contador ou sequência.



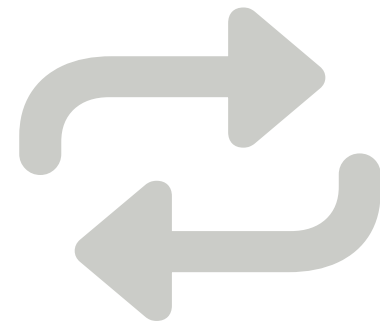
Laços de repetição

For

```
for (inicialização; condição; incremento)
```

```
for (int i = 1; i <= 3; i++) {  
    printf("Repetição número: %d\n", i);  
}
```

Agenda



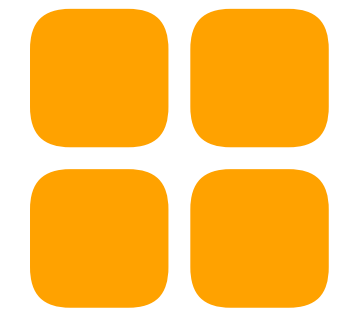
**Laços de
repetição**



Vetores

$f(x)$

Funções



Structs



Arrays - vetores

Um array é uma estrutura que permite armazenar vários valores do mesmo tipo usando uma única variável.

sem array:

```
int nota1;  
int nota2;  
int nota3;  
int nota4;  
int nota5;
```

Com array:

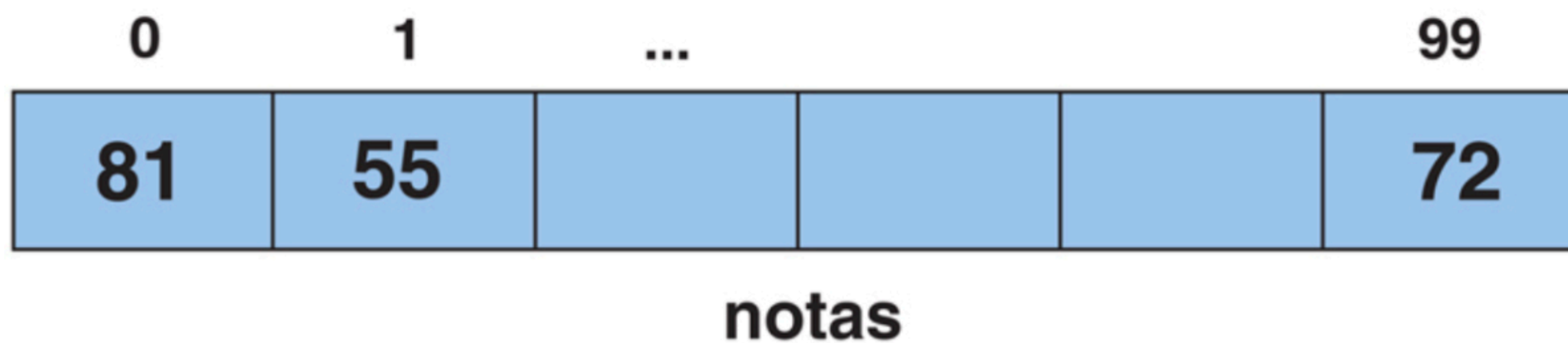
```
int notas[5];
```

Agora notas representa 5 posições de memória, todas do tipo int.



Vetores

posição “1” está armazenado o valor “55”, e assim por diante.





Vetores

O vetor é simplesmente um array de uma dimensão.

tipo_dado nome_array[tamanho];

```
int idades[10];  
  
float notas[5];  
  
char letras[26];  
  
double precos[100];
```



Vetores

```
#include <stdio.h>

int main() {
    int numeros[5]; // Declara um vetor de inteiros com 5 posições

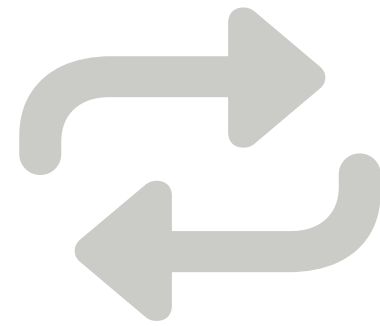
    // Preenchendo o vetor
    for (int i = 0; i < 5; i++) {
        printf("Digite o número %d: ", i + 1);
        scanf("%d", &numeros[i]); // Lê o número e armazena no vetor
    }

    // Imprimindo os números armazenados
    printf("Números digitados:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d\n", numeros[i]); // Imprime cada número
    }
}
```

Exemplo de Saída:

```
Números digitados:
10
20
30
40
50
```

Agenda



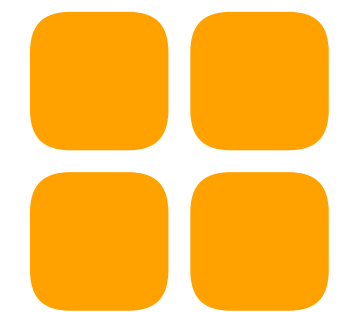
**Laços de
repetição**



Vetores

***f*(x)**

Funções



Structs

f(x) Funções

As funções em C são blocos de código que executam tarefas específicas e podem ser reutilizadas em diferentes partes do programa. Elas ajudam a organizar o código, melhorar a legibilidade e facilitar a manutenção.

$f(x)$ Funções

Estrutura de uma Função

- 1 **Tipo de Retorno:** Indica o tipo de dado que a função irá retornar (int, float, void, etc.).
- 2 **Nome da Função:** Identificador que descreve a função.
- 3 **Parâmetros:** Variáveis de entrada (opcionais) que a função pode receber.
- 4 **Corpo da Função:** Conjunto de instruções que define o que a função faz.

f(x) Funções

Exemplo

```
#include <stdio.h>

int somar(int a, int b) {
    return a + b;
}

int main() {
    int resultado;

    resultado = somar(5, 3);

    printf("Resultado: %d\n", resultado);

    return 0;
}
```

```
#include <stdio.h>

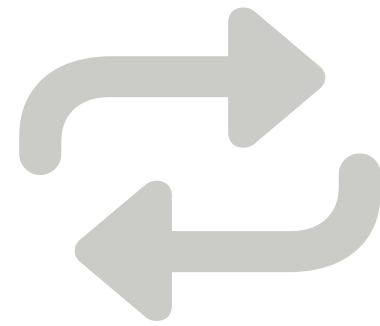
int somar(int a, int b); // protótipo

int main() {
    printf("%d", somar(5, 3));

    return 0;
}

int somar(int a, int b) {
    return a + b;
}
```

Agenda



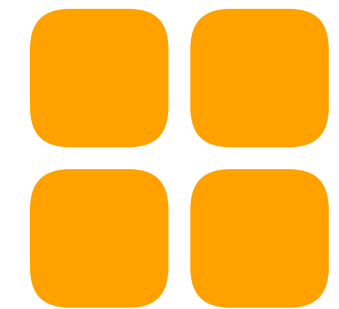
**Laços de
repetição**



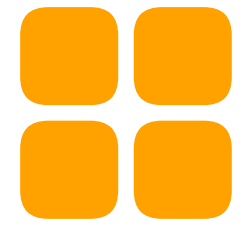
Vetores

$f(x)$

Funções



Structs



Structs

O que são?

Structs são estruturas que permitem agrupar diferentes tipos de dados sob um único nome. Isso é útil quando queremos representar um "objeto" que tem várias características.

Structs

EX: Criando a struct do tipo aluno:

```
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};
```

Structs

Criando a struct do tipo aluno:

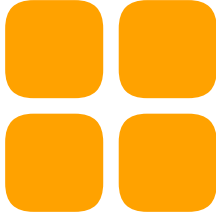
EX:

```
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};
```

Declarando e usando uma variável do tipo struct:

EX:

```
struct Aluno aluno1;  
  
strcpy (aluno1.nome, "Joao");  
aluno1.idade = 21;  
aluno1.nota = 9.5;  
  
Print("Nome: %s\n", aluno1.nome);  
Print("Idade: %s\n", aluno1.idade);  
Print("Nota: %s\n", aluno1.nota);
```



Structs

EX: Inicializando na declaração



```
struct Aluno aluno2 = {"João", 21, 8.0};
```

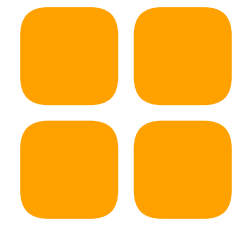
Structs

Usando typedef

Utilizado para não precisar digitar struct toda vez que for declarar uma variável:

EX:

```
struct Altypedef struct {  
    char nome[50];  
    int idade;  
    float nota;  
} Aluno;  
  
Aluno aluno1 = {"Joao", 18, 8.5};  
Aluno aluno2 = {"Pedro", 20, 9.5};  
Aluno aluno3 = {"Lucas", 21, 6.5};  
Aluno aluno2 = {"João", 21, 8.0};
```



Structs

Com typedef

```
typedef struct {  
    char nome[50];  
    int idade;  
    float nota;  
}Aluno;  
  
Aluno aluno1;  
Aluno aluno2;  
Aluno aluno3;
```

Sem typedef

```
struct Aluno{  
    char nome[50];  
    int idade;  
    float nota;  
};  
  
strcut Aluno aluno1;  
struct Aluno aluno2;  
struct Aluno aluno3;
```

Structs

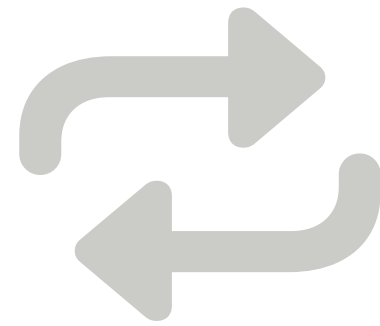
Vetor de structs

Assim como vetores de `int`, também podemos ter vetores de structs

EX:

```
Aluno turma[3] = {  
    {"Joao", 21, 8.5},  
    {"Joao", 22, 7.0},  
    {"Joao", 19, 9.5},  
};  
  
for (int i = 0; i < 3; i++) { printf("%s tem nota %.1f\n", turma[i].nome, turma[i].nota); }
```

Agenda



**Laços de
repetição**



Vetores

f(x)

Funções



Structs

Exercícios

Questão 1

Na elaboração de um algoritmo, a estrutura de repetição com teste no início (como o comando enquanto...faça) avalia a sua condição lógica

Alternativas:

- A)** Apenas no encerramento do programa.
- B)** Somente após a primeira execução do bloco de comandos.
- C)** Antes de cada execução do bloco de comandos.
- D)** Somente quando o contador atingir o valor máximo.

Questão 1

Na elaboração de um algoritmo, a estrutura de repetição com teste no início (como o comando enquanto...faça) avalia a sua condição lógica

Alternativas:

- A)** Apenas no encerramento do programa.
- B)** Somente após a primeira execução do bloco de comandos.
- C)** Antes de cada execução do bloco de comandos.
- D)** Somente quando o contador atingir o valor máximo.

Questão 2

A estrutura de dados array deve ser utilizada somente para a manipulação de strings.

Alternativas:

A) Certo

B) Errado

Questão 2

A estrutura de dados array deve ser utilizada somente para a manipulação de strings.

Alternativas:

A) Certo

B) Errado

Questão 3

```
int x = 5;  
while (x > 0) {  
    printf("%d ", x);  
    x--;  
}
```

Alternativas:

A) 5 4 3 2 1

B) 5 4 3 2 1 0

C) 4 3 2 1 0

D) 5 4 3 2

E) O código entra em loop infinito.

Questão 3

```
int x = 5;  
while (x > 0) {  
    printf("%d ", x);  
    x--;  
}
```

Alternativas:

A) 5 4 3 2 1

B) 5 4 3 2 1 0

C) 4 3 2 1 0

D) 5 4 3 2

E) O código entra em loop infinito.

Questão 4

```
int x = 10;  
do {  
    printf("%d ", x);  
    x++;  
} while (x < 5);
```

Alternativas:

- A)** Nada será impresso
- B)** 10
- C)** 10 11 12 13 14
- D)** O programa entra em loop infinito.
- E)** O código não compila.

Questão 4

```
int x = 10;  
do {  
    printf("%d ", x);  
    x++;  
} while (x < 5);
```

Alternativas:

A) Nada será impresso

B) 10

C) 10 11 12 13 14

D) O programa entra em loop infinito.

E) O código não compila.

Questão 5

```
int numeros[5] = {2, 4, 6, 8, 10};  
for (int i = 0; i < 5; i++) {  
    printf("%d ", numeros[i]);  
}
```

Alternativas:

- A)** O primeiro elemento acessado é numeros[1].
- B)** O último elemento acessado é numeros[5].
- C)** São acessados cinco elementos, dos índices 0 a 4.
- D)** São acessados seis elementos, dos índices 0 a 5.
- E)** O vetor possui índices de 1 a 5

Questão 5

```
int numeros[5] = {2, 4, 6, 8, 10};  
for (int i = 0; i < 5; i++) {  
    printf("%d ", numeros[i]);  
}
```

Alternativas:

- A)** O primeiro elemento acessado é numeros[1].
- B)** O último elemento acessado é numeros[5].
- C)** São acessados cinco elementos, dos índices 0 a 4.
- D)** São acessados seis elementos, dos índices 0 a 5.
- E)** O vetor possui índices de 1 a 5

Questão 6

```
int multiplicar(int a, int b) {  
    return a * b;  
}  
int resultado = multiplicar(4, 5);  
printf("%d", resultado);
```

Alternativas:

A) 9

B) 10

C) 20

D) 25

E) Nada será impresso.

Questão 6

```
int multiplicar(int a, int b) {  
    return a * b;  
}  
int resultado = multiplicar(4, 5);  
printf("%d", resultado);
```

Alternativas:

A) 9

B) 10

C) 20

D) 25

E) Nada será impresso.

Questão 7

```
void mensagem() {  
    printf("Olá!");  
}  
  
int main() {  
    mensagem();  
    return 0;  
}
```

Alternativas:

- A)** Ela obrigatoriamente precisa possuir um return.
- B)** Ela retorna uma string.
- C)** Ela não retorna um valor, pois seu tipo é void.
- D)** Ela retorna o valor 0 automaticamente.
- E)** Ela não pode ser chamada dentro da função main.

Questão 7

```
void mensagem() {  
    printf("Olá!");  
}  
  
int main() {  
    mensagem();  
    return 0;  
}
```

Alternativas:

- A)** Ela obrigatoriamente precisa possuir um return.
- B)** Ela retorna uma string.
- C)** Ela não retorna um valor, pois seu tipo é void.
- D)** Ela retorna o valor 0 automaticamente.
- E)** Ela não pode ser chamada dentro da função main.

Questão 8

Alternativas:

A) Maria será armazenada em idade.

B) 20 será armazenado em nome.

C) 9.5 será armazenado em nota.

D) A declaração apresenta erro porque uma struct não pode ser inicializada na declaração.

E) A struct só pode armazenar valores do mesmo tipo.

```
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};
```

```
struct Aluno aluno = {"Maria", 20, 9.5};
```

Questão 8

Alternativas:

A) Maria será armazenada em idade.

B) 20 será armazenado em nome.

C) 9.5 será armazenado em nota.

D) A declaração apresenta erro porque uma struct não pode ser inicializada na declaração.

E) A struct só pode armazenar valores do mesmo tipo.

```
struct Aluno {  
    char nome[50];  
    int idade;  
    float nota;  
};
```

```
struct Aluno aluno = {"Maria", 20, 9.5};
```

Obrigado a todos!

Contatos

